Plant Detector

On-device houseplant classifier · TinyML

Asaf Iron-Jobes · EE446 TinyML — Final Project

Arduino Nano 33 BLE Sense · OV7675 camera · MobileNetV1 · TensorFlow Lite Micro

Motivation and Why TinyML:

Motivation: Plant classification apps are quite popular but many require a paid subscription and all require internet. The idea for my project is to make a plant classification model that works offline and in low power.

Could be used for automated watering + plant care system

Extensions/adaptations of this idea could be used for wilderness survival device, agriculture, outdoors enthusiasts, and gardeners.

Why Tiny? – Free, low power, offline

OVERVIEW

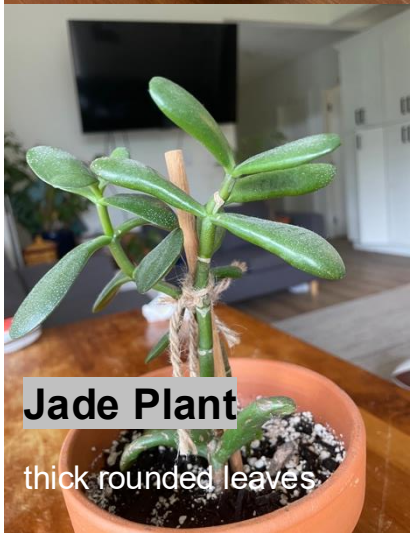
Classify 6 houseplants, entirely on the chip

- Live camera frame → species prediction + confidence on device → serial stream to python dashboard.
- Runs in 256 KB RAM / 1 MB flash on the Nano 33 BLE Sense (nRF52840, Cortex-M4).
- Streams the prediction + the model's-eye image to a desktop dashboard over USB serial.
- Specialized to the plants, camera, and environment used in the demo.


Aloe Vera
succulent, spiky rosette


Asparagus Fern
fine feathery fronds


Dumb Cane
broad variegated leaves

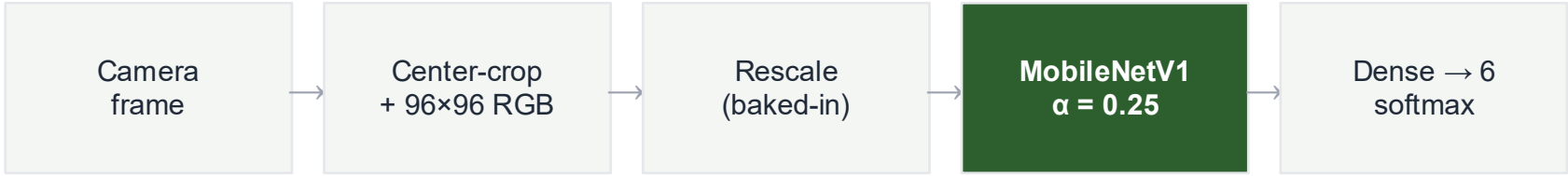

Jade Plant
thick rounded leaves


Money Tree
palmate 5-leaflet


Snake Plant
tall stiff striped blades

MODEL ARCHITECTURE

MobileNetV1 + transfer learning, quantized to INT8



- Transfer learning: ImageNet-pretrained MobileNetV1 backbone + a small new head (global-average-pool → dropout → dense-6).
- Normalization (0–255 → −1…1) is baked in as the first layer, so the device feeds raw camera pixels.
- Post-training INT8 quantization for the deployed model; runs under TensorFlow Lite Micro.

DECISIONS & TRADEOFFS

Design choices

MobileNetV1, not V2/V3

V2/V3's channel expansion needs ~293 KB of activations — over budget. V1 has no expansion → ~103 KB. Fits.

96×96 input

Higher resolution than the 64/48. used V1 to allow for higher res

Post-training INT8 (not QAT)

Lossless here (0.82→0.82), ~4× smaller, integer math on the MCU. Simpler than QAT for no accuracy cost.

5-frame peak-hold + threshold

Rewards the best-aimed frame and rejects low-confidence views → a stable, honest demo.

TFLM + plain Python, not Edge Impulse

Full control of the firmware loop and a readable, debuggable pipeline.

DATA

Three sources, general → deployment-specific

Online (Kaggle)

~2,000 imgs

General plant features

Phone camera

my plants

My actual plants, sharper than the sensor.

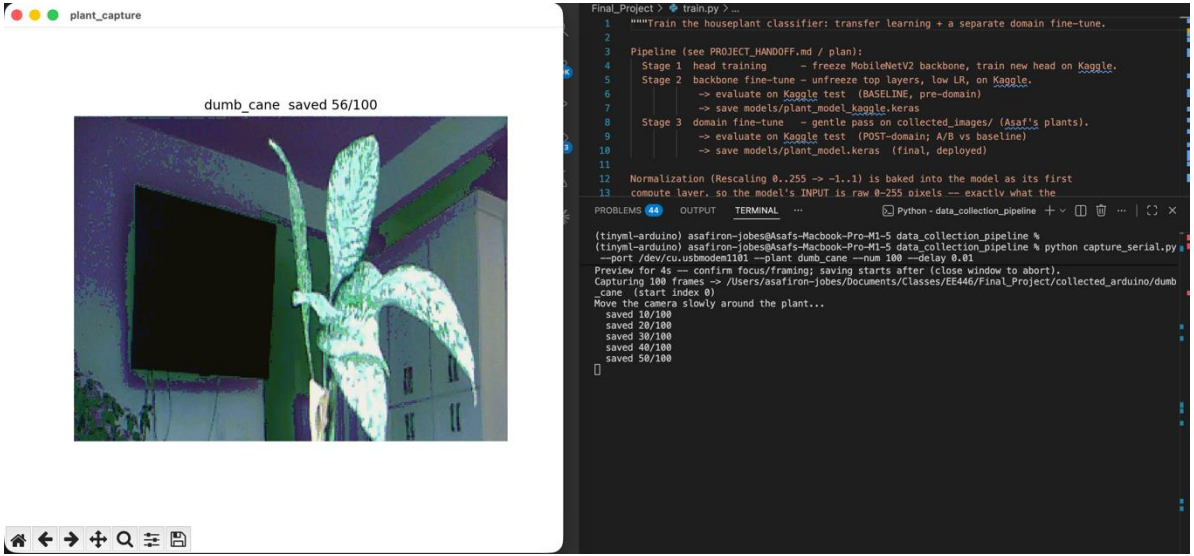
Arduino camera

600 frames

Captured through the OV7675 on the demo deployment domain.

Capturing environment specific data on Arducam

- Custom Arduino sketch streams OV7675 frames over USB serial (request/response, sync-header framed).
- Python tool saves frames into per-class folders with a live preview to aim and focus the lens.
- 100 frames/plant, moving around each plant on the demo table → realistic, on-device-matched images.



ONLINE & PHONE DATA

Building (and cleaning) the base dataset

Online — Kaggle House Plant Species

- 6 of 47 classes selected to match my plants.
- EXIF rotation fixed so images aren't fed sideways.

Phone — my own plants

- Photos of the actual plants I own.
- Merged into the main dataset as ordinary training data.
- Degrade-augmentation (blur, color cast, noise) makes crisp photos look like the grainy sensor feed.

TRAINING STRATEGY

Two stages: general training → on-device fine-tuning

STAGE 1

General training

Online + phone images

- Pretrained MobileNetV1 backbone (ImageNet transfer learning).
- Train the new 6-class head, then unfreeze the top layers and fine-tune at low LR.
- Learns what the six species look like in general.

→

STAGE 2

Environment-specific fine-tuning

Arduino-camera data

- Fine-tune on frames captured through the OV7675 on the demo table.
- Adapts to real deployment: low-res sensor, lighting, distance.
- Rehearsal + early-stopping → specialize without forgetting.

RESULTS

Float vs INT8

Variant	Accuracy (cam)	Model size	Latency*	Tensor arena
Float (reference)	0.82	849 KB	0.27 ms	—
INT8 (deployed)	0.82	303 KB	0.14 ms	~103 KB

- INT8 quantization was lossless on the camera test set, at ~2.8× smaller.
- ~103 KB arena measured on-device
- Held-out Arduino test ≈ 0.82; the real measure is the live demo.

RESULTS

Training & test accuracy

Accuracy through the pipeline

Training phase	Val	General test	Camera test
Stage 1 — General training (online + phone)	77%	76.9%	23.3%
Stage 2 — Camera fine-tuning (Arduino)	85%	69.4%	**81.7%**

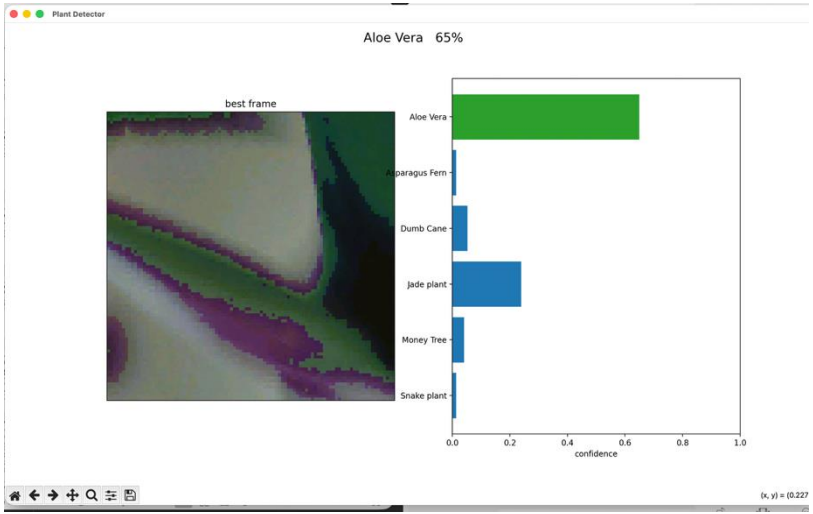
Deployed model (quantization)

Model	Camera test	General test	Size	Latency*
Float (reference)	81.7%	69.4%	849 KB	0.27 ms
INT8 (on-device)	**81.7%**	71.5%	**303 KB**	0.14 ms

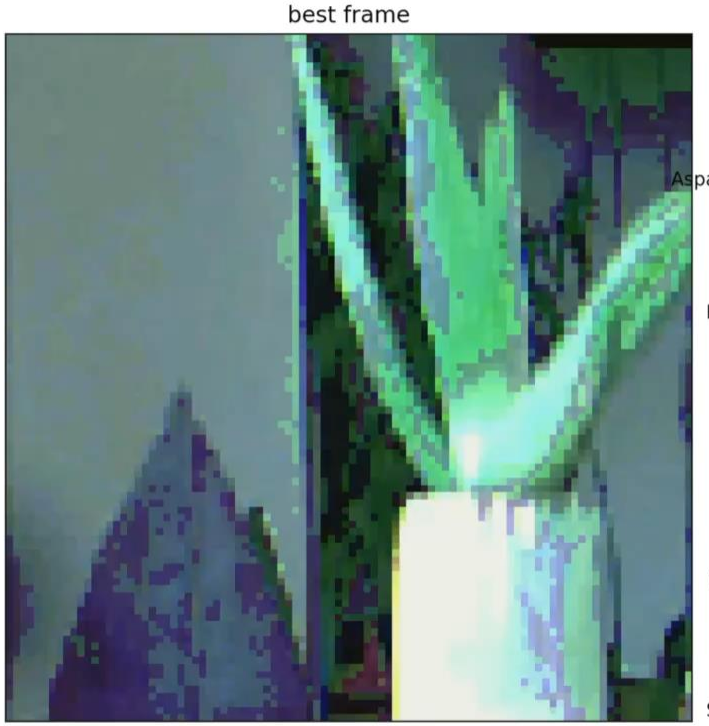
DEPLOYMENT & DASHBOARD

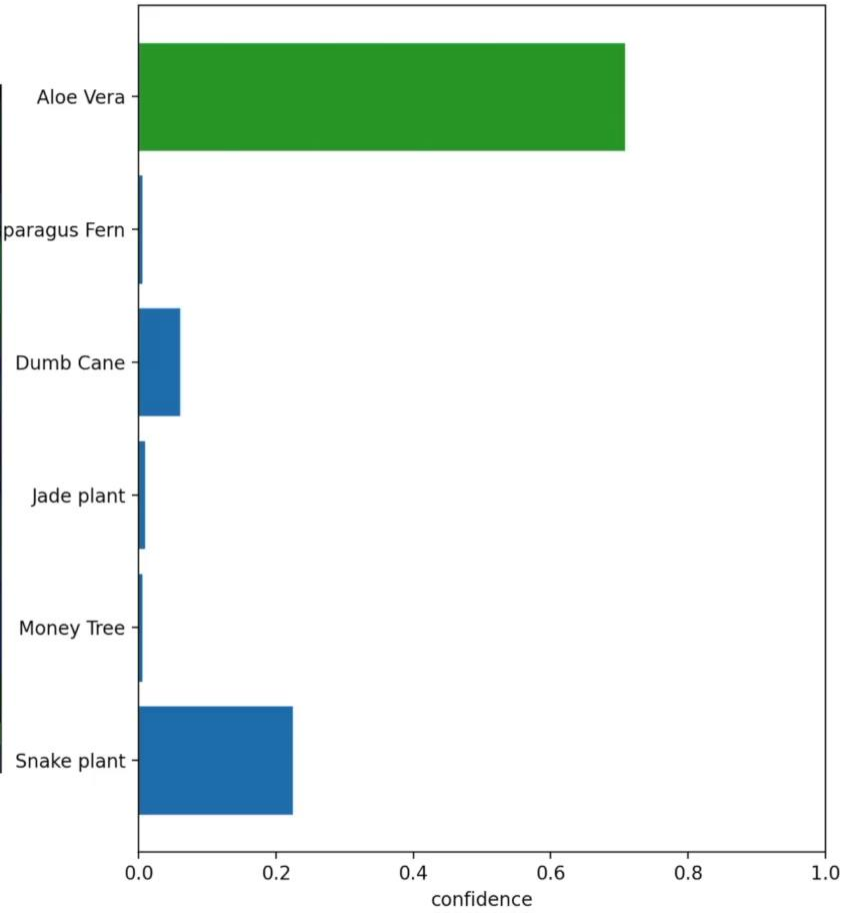
On-device loop + live serial UI

- Device loop: capture → center-crop/resize → INT8 inference → 5-frame peak-hold + → serial.
- Outputs the most confident recent prediction, or "No houseplant detected" below threshold.
- Python dashboard shows the model's-eye image and live 6-class confidence bars.
- Pipelined on the mbed RTOS — a dedicated TX thread overlaps serial transmission with the main thread's capture + inference (semaphore-synced).



Aloe Vera 86%


best frame


Aloe Vera
Asparagus Fern
Dumb Cane
Jade plant
Money Tree
Snake plant
0.0
0.2
0.4
0.6
0.8
1.0
confidence

APPENDIX

References

Dataset

- “House Plant Species” image dataset — Kaggle.

Models & papers

- Howard et al., “MobileNets: Efficient CNNs for Mobile Vision Applications,” arXiv:1704.04861, 2017.
- Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR 2018 (arXiv:1801.04381).
- ImageNet — pretrained backbone weights.

Frameworks & tools

- TensorFlow / Keras — model training.
- TensorFlow Lite + TFLite for Microcontrollers — INT8 quantization & on-device inference.
- Arduino IDE + Arduino Mbed OS Nano core.

Hardware

- Arduino Nano 33 BLE Sense (Nordic nRF52840).
- Arducam OV7675 0.3 MP camera module.

Libraries & reused code

- Arduino_TensorFlowLite library.
- Harvard TinyMLx — TinyMLShield / Arduino_OV767X; camera + TFLM examples adapted from EE446 Lab 5 (CameraCaptureRawBytes, person_detection).
- Python: NumPy, Pillow, pySerial, Matplotlib.